OAVS • TGT COMPUTER SCIENCE

MCQ Question Bank

Topic: Functions

Subprograms · Interfaces · Arguments · Return Statements · Local Variables · void · Recursion · Call by Value & Reference

165	16	100%
MCQS	CHAPTERS	SYLLABUS ALIGNED

Exam-Oriented • Answer Key + Explanations Included

Table of Contents

1	1. Concept of a Subprogram (Function)	Q1–Q12
2	2. Interface of a Subprogram	Q13–Q22
3	3. Role of an Interface	Q23–Q30
4	4. Arguments of a Subprogram	Q31–Q42
5	5. Kinds of Subprograms Supported by C	Q43–Q52
6	6. Return Statement as an Interface	Q53–Q62
7	7. Local Variables	Q63–Q72
8	8. Default Return Type and void	Q73–Q82
9	9. Passing Values Between Functions Through Interfaces	Q83–Q89
10	10. Declaration of Function Type	Q90–Q99
11	11. Iterative and Recursive Subprograms	Q100–Q107
12	12. Recursion	Q108–Q118
13	13. Call by Value	Q119–Q130
14	14. Call by Reference	Q131–Q142
15	15. Return and Their Underlying Implementation	Q143–Q149
16	16. Similarities and Comparisons	Q150–Q165

Chapter 1

1. Concept of a Subprogram (Function)

12 Questions

Q1 Chapter 1 — Definition

What is a subprogram in C also commonly called?

- **A** A macro
- **B** A statement
- **C** A header file
- **D** A function

Ans: D

EXPLANATION

A subprogram is a self-contained block of code designed to perform a specific task; in C it is called a function.

Q2 Chapter 1 — Purpose

What is the primary purpose of using functions in a program?

- **A** To remove the need for variables
- **B** To slow down program execution
- **C** To break a large program into smaller, manageable, reusable tasks
- **D** To increase the size of the source file

Ans: C

EXPLANATION

Functions divide a program into logical, reusable units, making code easier to write, understand, and maintain.

Q3 Chapter 1 — Characteristics

Which of the following is a key characteristic of a well-designed function?

- **A** It must always return a value
- **B** It must be written before main()
- **C** It cannot accept parameters
- **D** It performs a single, well-defined task

Ans: D

EXPLANATION

A good function is designed around a single, clearly defined task, which improves readability and reusability.

Q4 Chapter 1 — Advantages

Which of these is an advantage of modular programming using functions?

- **A** Increased code duplication
- **B** Reduced program readability
- **C** Easier debugging and code reuse
- **D** Mandatory global variables

Ans: C

EXPLANATION

Modular programming allows code to be reused and tested independently, which simplifies debugging.

Q5 Chapter 1 — Applications

Functions are most useful when a particular task needs to be performed:

- **A** Only once at the very end of the program
- **B** Only inside the main() function
- **C** Only when no arguments are needed
- **D** Repeatedly at multiple points in a program

Ans: D

EXPLANATION

When the same task is needed multiple times, writing it as a function avoids rewriting the same code repeatedly.

Q6 Chapter 1 — Definition

A subprogram that performs a specific, isolated task and can be invoked from other parts of a program is called a:

- **A** Loop
- **B** Comment block
- **C** Preprocessor directive
- **D** Function

Ans: D

EXPLANATION

This is the standard definition of a function/subprogram in structured programming.

Q7 Chapter 1 Characteristics

Which characteristic allows a function to be used in different programs without modification?

A Fixed hard-coded values
B Global variable dependency
C Modularity and independence
D Direct hardware access

Ans: C

EXPLANATION

A modular, self-contained function with no unnecessary dependencies can be reused across programs.

Q8 Chapter 1 Advantages

Modular programming primarily improves which quality of a program?

A File size on disk
B Screen resolution
C Maintainability
D Number of compiler warnings

Ans: C

EXPLANATION

Breaking a program into modules (functions) makes it easier to maintain, update, and debug.

Q9 Chapter 1 Purpose

Why do large programs benefit most from being divided into functions?

A It reduces complexity and improves clarity
B It increases compilation time only
C It automatically removes all bugs
D It prevents any function calls

Ans: A

EXPLANATION

Dividing a large program into functions reduces overall complexity and makes the logic easier to follow.

Q10 Chapter 1 Applications

Which of the following is a typical real-world application of using functions?

A Avoiding the use of the main() function

B Writing the entire program in a single line

C Calculating the same formula for multiple inputs

D Declaring only one variable in the whole program

Ans: C

EXPLANATION

Functions are ideal when the same computation needs to be applied to different inputs across a program.

Q11 Chapter 1 Characteristics

A function in C must have:

A At least one global variable

B Only a return statement

C A mandatory loop inside it

D A name and a body enclosed in braces

Ans: D

EXPLANATION

Every C function has a name and a body (defined within braces) containing the statements to execute.

Q12 Chapter 1 Purpose

Which statement best explains the concept of a subprogram?

A It is a comment used for documentation

B It is a type of variable declaration

C It is a separate executable file that runs independently of the main program

D It is a smaller program within a program that solves part of a larger problem

Ans: D

EXPLANATION

A subprogram is conceptually a smaller unit of code that helps solve part of a larger problem within the same program.

Chapter 2 — 2. Interface of a Subprogram — 10 Questions

Q13 Chapter 2 — Meaning

What does the term 'function interface' refer to?

- **A** The number of lines in the function
- **B** The physical location of the function in memory
- **C** The way a function communicates with the rest of the program
- **D** The color scheme of the code editor

Ans: C

EXPLANATION

The function interface defines how a function interacts with the caller, i.e., how data is passed in and results are passed out.

Q14 Chapter 2 — Components

Which of the following is NOT a component of a function interface?

- **A** Parameter list
- **B** Return type
- **C** Function name
- **D** The compiler version used

Ans: D

EXPLANATION

The function interface consists of the function name, parameters, and return type — not the compiler version.

Q15 Chapter 2 — Components

The part of a function interface that specifies what kind of value a function sends back is called the:

- **A** Return type
- **B** Function body
- **C** Loop counter
- **D** Local variable

Ans: A

EXPLANATION

The return type in the function interface tells the caller what type of value to expect from the function.

Q16 Chapter 2 — Function name

In a function interface, the function name serves to:

- **A** Replace the need for parameters
- **B** Uniquely identify the function when it is called
- **C** Store the return value permanently
- **D** Define the number of loops inside the function

Ans: B

EXPLANATION

The function name is used by the caller to identify and invoke the correct function.

Q17 Chapter 2 — Parameters

Parameters in a function interface are used to:

- **A** Store the function's compiled machine code
- **B** Automatically print output
- **C** Define the operating system used
- **D** Pass data into the function from the caller

Ans: D

EXPLANATION

Parameters allow the caller to send input data into the function for processing.

Q18 Chapter 2 — Interaction

The function interface primarily governs interaction between:

- **A** Two unrelated programs on the same computer
- **B** The keyboard and the monitor
- **C** The calling function and the called function
- **D** The compiler and the linker only

Ans: C

EXPLANATION

The interface defines exactly how the caller and the called function exchange information.

Q19 Chapter 2 Meaning

Which of the following best describes the 'interface' of a function?

A The number of times the function is called

B The comments written inside the function

C The complete internal implementation of the function

D Its name, parameter list, and return type, without including its internal logic

Ans: D

EXPLANATION

The interface is the external view of a function — what it is called, what it takes in, and what it returns — hiding internal logic.

Q20 Chapter 2 Return type

If a function's interface specifies a return type of int, the function is expected to:

A Return an integer value to the caller

B Never return any value

C Return a character string

D Accept only integer parameters

Ans: A

EXPLANATION

The return type in the interface tells the caller the data type of the value the function sends back.

Q21 Chapter 2 Interaction

Why is a clearly defined function interface important in programming?

A It ensures the caller and called function agree on data types and communication rules

B It forces every function to use the same variable names

C It eliminates the need to compile the program

D It reduces the number of functions a program can have

Ans: A

EXPLANATION

A well-defined interface avoids confusion by clearly specifying how data will be exchanged between caller and function.

Q22 Chapter 2 Components

Which three elements together typically form a function's interface in C?

A Function name, parameter list, and return type

B File name, line number, and column number

C Loop variables, arrays, and pointers

D Function name, comments, and indentation

Ans: A

EXPLANATION

Together, the function name, parameters, and return type define the interface of the function.

Chapter 3 — 3. Role of an Interface — 8 Questions

Q23 Chapter 3 — Purpose

What is the main role of an interface in a function?

- **A** To increase the execution time of the program
- **B** To prevent the function from being called more than once
- **C** To hide the function name from the user
- **D** To define a clear contract for communication between caller and function

Ans: D

EXPLANATION

The interface establishes a contract describing what inputs are needed and what output will be produced.

Q24 Chapter 3 — Communication

The interface of a subprogram enables communication by specifying:

- **A** How many times the program has been compiled
- **B** The physical size of the computer's memory
- **C** What data goes in and what result comes out
- **D** Which programmer wrote the function

Ans: C

EXPLANATION

The interface specifies the inputs (parameters) and outputs (return value), enabling communication between functions.

Q25 Chapter 3 — Encapsulation

How does a function interface support encapsulation?

- **A** By making all variables global
- **B** By hiding the internal implementation details from the caller
- **C** By exposing all local variables to other functions
- **D** By printing the function's source code automatically

Ans: B

EXPLANATION

Encapsulation is supported because the caller only needs to know the interface, not how the function works internally.

Q26 Chapter 3 Modularity

The role of an interface in supporting modularity is to:

A Force all functions to be written in a single file

B Prevent functions from having return values

C Allow functions to be developed and modified independently

D Require every function to call every other function

Ans: C

EXPLANATION

A clear interface allows a function's internal code to change without affecting other parts of the program, supporting modularity.

Q27 Chapter 3 Data exchange

Interfaces facilitate data exchange between functions mainly through:

A The compiler's error messages

B The program's file name

C Comments and blank lines

D Parameters and return values

Ans: D

EXPLANATION

Data flows between functions via parameters (input) and the return statement (output), both defined in the interface.

Q28 Chapter 3 Purpose

Which of the following best explains why interfaces are important for large software projects?

A They eliminate the need for testing

B They remove the need for a main() function

C They allow different programmers to work on different functions independently, as long as the interface is respected

D They allow only one function to exist in the entire project

Ans: C

EXPLANATION

In large projects, as long as the agreed interface is followed, functions can be developed independently by different team members.

Q29 Chapter 3 Encapsulation

A function whose internal working is hidden from the caller, exposing only its interface, is an example of:

A Type casting

B Recursion

C Encapsulation

D Global scoping

Ans: C

EXPLANATION

Encapsulation means the internal details are hidden, and only the interface (name, parameters, return type) is visible to the user.

Q30 Chapter 3 Communication

If two functions need to exchange data, the safest and most structured way to do this is through:

A The function interface (parameters and return values)

B Direct memory manipulation without any structure

C Randomly naming variables the same in both functions

D Printing values to the screen and re-typing them

Ans: A

EXPLANATION

The function interface provides a structured, safe mechanism for exchanging data between functions.

Chapter 4 — 4. Arguments of a Subprogram — 12 Questions

Q31 Chapter 4 — Actual arguments

Actual arguments are the values that are:

- **A** Never used during a function call
- **B** Passed to a function at the time it is called
- **C** Always equal to zero
- **D** Declared inside the function definition only

Ans: B

EXPLANATION

Actual arguments are the real values supplied by the caller when the function is invoked.

Q32 Chapter 4 — Formal arguments

Formal arguments are:

- **A** Variables declared in the function definition that receive the actual arguments
- **B** Only used in recursive functions
- **C** Constants defined outside all functions
- **D** Values printed on the screen after the function executes

Ans: A

EXPLANATION

Formal parameters are the variables listed in the function header/definition that receive the values passed by the caller.

Q33 Chapter 4 — Parameter passing

During a function call, the process by which actual arguments are transferred to formal arguments is called:

- **A** Parameter passing
- **B** Type casting
- **C** Recursion
- **D** Compilation

Ans: A

EXPLANATION

Parameter passing is the mechanism used to transfer values from actual arguments to formal parameters.

Q34 Chapter 4 Argument matching

In C, actual arguments must match formal arguments in:

A Variable name only

B Font style used in the code editor

C Number and, generally, compatible data type

D Memory address only

Ans: C

EXPLANATION

The number of arguments and their types must correspond between the actual and formal arguments (with allowed conversions).

Q35 Chapter 4 Characteristics

Which statement about formal arguments is TRUE?

A They exist for the entire duration of the program

B They can be accessed from any function without restriction

C They exist only within the scope of the function in which they are declared

D They must always be declared as global variables

Ans: C

EXPLANATION

Formal arguments are local to the function; their scope is limited to the function body.

Q36 Chapter 4 Uses

Arguments are primarily used in functions to:

A Increase the number of lines in a program

B Replace the return statement

C Automatically generate comments

D Supply input data required for the function to perform its task

Ans: D

EXPLANATION

Arguments allow a function to receive the specific input data it needs to carry out its task.

Q37 Chapter 4 Actual arguments

In the call add(5, 10);, the values 5 and 10 are known as:

A Formal arguments
B Actual arguments
C Local variables
D Return values

Ans: B

EXPLANATION

The values supplied directly in the function call, here 5 and 10, are called actual arguments.

Q38 Chapter 4 Argument matching

What happens if the number of actual arguments does not match the number of formal parameters in a C function (with no default values)?

A It results in a compilation error
B The extra formal parameters are automatically set to 100
C The program ignores the mismatch and runs normally
D The function is automatically converted to void

Ans: A

EXPLANATION

C requires the number of actual and formal arguments to match; mismatches typically cause a compile-time error.

Q39 Chapter 4 Parameter passing

Which of the following is a valid method of parameter passing supported in C?

A Call by object
B Call by value
C Call by interface
D Call by name only

Ans: B

EXPLANATION

C primarily supports call by value directly; call by reference is simulated using pointers.

Q40 Chapter 4 Formal arguments

Formal parameters are created in memory:

A Only if the function is recursive

B Before the program starts and never destroyed

C Only when the program ends

D When the function is called and destroyed when it returns

Ans: D

EXPLANATION

Formal parameters are local variables allocated when the function is invoked and deallocated once the function returns.

Q41 Chapter 4 Uses

Why might a function require multiple arguments?

A To avoid using a return statement

B To make the function harder to call

C To provide all the different pieces of input data the function needs to complete its task

D Because C does not allow single-argument functions

Ans: C

EXPLANATION

Multiple arguments let a function receive several pieces of related input data at once.

Q42 Chapter 4 Characteristics

Changing the value of a formal parameter inside a function, when using call by value, affects:

A Every variable in the program

B The original actual argument in the calling function

C Only the local copy inside the function

D The return type of the function

Ans: C

EXPLANATION

In call by value, formal parameters are independent local copies, so changes do not affect the caller's original variables.

Chapter 5 — 5. Kinds of Subprograms Supported by C — 10 Questions

Q43 Chapter 5 — Library functions

Functions such as printf() and scanf() that are pre-written and provided with the C compiler are called:

- **A** Void functions
- **B** User-defined functions
- **C** Recursive functions
- **D** Library functions

Ans: D

EXPLANATION

Library functions are pre-built functions supplied as part of the standard C library, ready to use via header files.

Q44 Chapter 5 — User-defined

A function that is written by the programmer to perform a task specific to their program is called a:

- **A** Macro
- **B** User-defined function
- **C** System call
- **D** Library function

Ans: B

EXPLANATION

User-defined functions are created by the programmer to handle tasks specific to the application being written.

Q45 Chapter 5 — With arguments

A function that receives data from the caller through its parameter list is classified as a function:

- **A** Without arguments
- **B** With arguments
- **C** That cannot be called
- **D** Without a return type

Ans: B

EXPLANATION

Functions with arguments accept input values through their parameter list when called.

Q46 Chapter 5 Without arguments

A function defined as void display(void) is an example of a function:

A That must be recursive

B Without arguments and without a return value

C That is part of the C standard library only

D With arguments and with a return value

Ans: B

EXPLANATION

The empty parentheses (or void) and void return type indicate this function takes no arguments and returns nothing.

Q47 Chapter 5 With return value

A function that sends a computed result back to the calling function using the return statement is a function:

A That is always recursive

B Without a return value

C Without arguments only

D With a return value

Ans: D

EXPLANATION

Such functions use 'return' to send a value back to the point where they were called.

Q48 Chapter 5 Without return value

Functions declared with a void return type are classified as functions:

A With a mandatory return value

B That cannot accept arguments

C Without a return value

D That are automatically recursive

Ans: C

EXPLANATION

A void return type means the function does not send any value back to the caller.

Q49 Chapter 5 Library functions

Which header file must typically be included to use the standard library function sqrt()?

A conio.h
B math.h
C stdio.h
D string.h

Ans: B

EXPLANATION

The sqrt() function is declared in the math.h header file of the C standard library.

Q50 Chapter 5 User-defined

One key benefit of writing user-defined functions is that they:

A Always execute faster than library functions
B Cannot accept any parameters
C Must be written only in recursive form
D Can be tailored exactly to the specific needs of the program

Ans: D

EXPLANATION

Since the programmer writes them, user-defined functions can be designed precisely for the program's requirements.

Q51 Chapter 5 With arguments

Which of the following function prototypes represents a function with arguments?

A void greet();
B void show(void);
C int getValue(); // no parameters used
D int add(int a, int b);

Ans: D

EXPLANATION

int add(int a, int b); clearly declares two parameters, making it a function with arguments.

Q52 Chapter 5 With return value

Which of the following function declarations indicates a function that returns a value?

A void print(int x);

B void show(char c);

C void display(void);

D float area(float r);

Ans: D

EXPLANATION

The return type float (instead of void) indicates that this function returns a value.

Chapter 6 6. Return Statement as an Interface 10 Questions

Q53 Chapter 6 Purpose

The main purpose of the return statement in a function is to:

- **A** Send a value back to the caller and/or terminate the function
- **B** Start the execution of main()
- **C** Import a header file
- **D** Declare a new function

Ans: A

EXPLANATION

The return statement both ends the function's execution and optionally sends a value back to the calling code.

Q54 Chapter 6 Syntax

Which of the following is the correct syntax to return the value of variable 'sum' from a function?

- **A** return sum;
- **B** returning sum;
- **C** return sum();
- **D** return: sum;

Ans: A

EXPLANATION

The correct C syntax is 'return' followed by the expression or variable, ending with a semicolon.

Q55 Chapter 6 Returning values

If a function is declared with return type int, the expression in its return statement should evaluate to:

- **A** A function name
- **B** An integer (or a value convertible to int)
- **C** Nothing at all
- **D** A string literal only

Ans: B

EXPLANATION

The value in the return statement should match, or be convertible to, the function's declared return type.

Q56 Chapter 6 Returning control

Besides sending a value, the return statement also:

A Transfers control back to the point where the function was called

B Pauses the program indefinitely

C Deletes the function from memory permanently

D Restarts the entire program

Ans: A

EXPLANATION

Execution control returns to the calling function immediately after a return statement executes.

Q57 Chapter 6 Multiple return scenarios

Can a function contain more than one return statement?

A Yes, but only one of them will execute during a particular function call

B No, C allows only a single return statement in the entire function

C Yes, and all of them always execute in one call

D No, this causes a syntax error even if reachable

Ans: A

EXPLANATION

A function can have multiple return statements (e.g., inside if-else branches), but only one executes per call, based on the flow of control.

Q58 Chapter 6 Function termination

What happens to the execution of a function as soon as a return statement is executed?

A The function restarts from the beginning

B The function pauses and waits for user input

C The rest of the function continues to execute normally

D The function terminates immediately, skipping remaining statements

Ans: D

EXPLANATION

Once 'return' executes, the function exits immediately; any statements after it in that execution path are skipped.

Q59 Chapter 6 Purpose

In a void function, the return statement (without any value) is used to:

A Cause a compilation error

B Convert the function into a non-void function

C Send back the value zero automatically

D Exit the function early, without sending any value back

Ans: D

EXPLANATION

A bare 'return;' in a void function simply exits the function early; it does not send a value.

Q60 Chapter 6 Returning values

What is a likely consequence of not including a return statement in a function declared to return int?

A The function is automatically treated as void

B The function's returned value is undefined/ unreliable behavior may occur

C The program will not compile under any circumstances

D The compiler automatically returns 100

Ans: B

EXPLANATION

Omitting 'return' in a non-void function leads to undefined behavior since no valid value is guaranteed to be sent back.

Q61 Chapter 6 Function termination

Which of the following best completes: 'The return statement acts as the -----.'

A starting point of the entire program

B declaration of a new variable

C definition of a new data type

D exit point of a function that also carries data back to the caller

Ans: D

EXPLANATION

The return statement is both the exit point of the function and the mechanism for sending data back to the caller.

Q62 Chapter 6 Syntax

Which return statement is syntactically valid inside a function with return type void?

A void return;

B return 0 void;

C return;

D return void;

Ans: C

EXPLANATION

In a void function, 'return;' with no value is valid; a value cannot be returned from a void function.

Chapter 7 — 7. Local Variables — 10 Questions

Q63 Chapter 7 — Definition

A local variable in C is a variable that is:

- **A** Declared outside all functions and accessible everywhere
- **B** Declared using the keyword global
- **C** Declared inside a function or block and accessible only within it
- **D** Automatically shared between all functions

Ans: C

EXPLANATION

Local variables are declared within a function/block and can only be accessed within that function/block.

Q64 Chapter 7 — Scope

The scope of a local variable is limited to:

- **A** The function or block in which it is declared
- **B** The entire program, including all files
- **C** Every function that is called after it
- **D** Only the main() function, regardless of where it is declared

Ans: A

EXPLANATION

A local variable's scope is restricted to the function or block where it was declared.

Q65 Chapter 7 — Lifetime

The lifetime of an ordinary local variable (not static) lasts:

- **A** Until another function is called anywhere in the program
- **B** Only while the function in which it is declared is executing
- **C** Until the computer is restarted
- **D** For the entire duration of the program

Ans: B

EXPLANATION

Local variables are created when the function starts executing and destroyed when the function returns, unless declared static.

Q66 Chapter 7 Visibility

A local variable declared inside function A is:

A Visible only inside main()

B Automatically visible inside every other function

C Not visible or accessible inside function B

D Visible only if declared with the keyword extern

Ans: C

EXPLANATION

Since local variables are scoped to their own function, they cannot be directly accessed by other functions.

Q67 Chapter 7 Characteristics

Which of the following is a characteristic of local variables?

A They must always be initialized to zero automatically

B They are shared across all functions by default

C They are typically stored in a function's own stack frame

D They exist even before the program starts running

Ans: C

EXPLANATION

Local variables are typically allocated on the stack, within the function's own stack frame, when the function is called.

Q68 Chapter 7 Applications

Local variables are most appropriate for storing:

A The name of the source code file

B Values that must be shared across the entire program

C Data that should persist after the program ends

D Temporary values needed only during the execution of a specific function

Ans: D

EXPLANATION

Local variables are ideal for temporary computations that are only relevant within a single function call.

Q69 Chapter 7 Scope

If two different functions each declare a local variable with the same name, what happens?

A Each variable is treated independently with no conflict, as their scopes do not overlap

B The program automatically renames one variable

C A compilation error always occurs due to duplicate names

D Both variables share the exact same memory location

Ans: A

EXPLANATION

Because each variable's scope is limited to its own function, using the same name in different functions causes no conflict.

Q70 Chapter 7 Lifetime

What happens to the memory of a local variable once its function finishes executing?

A It is permanently saved to disk

B It is typically deallocated/released

C It becomes a global variable

D It is copied automatically into main()

Ans: B

EXPLANATION

Once the function returns, the memory used by its local (automatic) variables is generally released.

Q71 Chapter 7 Definition

Local variables are also sometimes referred to as:

A External variables

B Static variables

C Register-only variables

D Automatic variables

Ans: D

EXPLANATION

In C, ordinary local variables are also called automatic variables because storage is automatically created and released.

Q72 Chapter 7 Visibility

To make a value calculated inside a function available to another function, a programmer typically should:

A Declare the variable local to a third, unrelated function

B Simply reference it by name in the other function without any mechanism

C Return the value or pass it as an argument, since local variables are not directly visible elsewhere

D Rename the variable to match the other function's name

Ans: C

EXPLANATION

Since local variables aren't visible outside their function, data must be shared explicitly via return values or parameters.

Chapter 8 — 8. Default Return Type and void — 10 Questions

Q73 Chapter 8 — Default return type

In older C standards, if no return type is specified for a function, what is assumed by default?

- **A** void
- **B** char
- **C** int
- **D** float

Ans: C

EXPLANATION

Traditionally in C, if a function's return type is omitted, the compiler assumes it to be int by default.

Q74 Chapter 8 — Meaning of void

The keyword void, when used as a function's return type, indicates that the function:

- **A** Can only be called once
- **B** Does not return any value
- **C** Must contain a loop
- **D** Always returns zero

Ans: B

EXPLANATION

void as a return type means the function performs its task without sending any value back to the caller.

Q75 Chapter 8 — Functions returning void

Which of the following function headers correctly declares a function that returns nothing?

- **A** printMessage(void) void;
- **B** void printMessage(int x);
- **C** int printMessage(void);
- **D** void printMessage() return 0;

Ans: B

EXPLANATION

void printMessage(int x); correctly shows a void return type with a parameter, meaning it returns nothing.

Q76 Chapter 8 — Void parameters

Writing void inside a function's parentheses, as in int getValue(void), specifies that the function:

- **A** Must be called recursively
- **B** Is a library function
- **C** Returns no value
- **D** Takes no arguments

Ans: D

EXPLANATION

void inside the parentheses of a function declaration explicitly states that the function accepts no parameters.

Q77 Chapter 8 — Characteristics

Which statement about void functions is TRUE?

- **A** They can never accept any parameters
- **B** They cannot be called from main()
- **C** They are automatically treated as recursive functions
- **D** They can still accept parameters even though they don't return a value

Ans: D

EXPLANATION

A void function can accept parameters for input, even though it does not return any value as output.

Q78 Chapter 8 — Applications

A function used only to display output on the screen, without producing any value to be used elsewhere, is typically declared with return type:

- **A** int
- **B** float
- **C** double
- **D** void

Ans: D

EXPLANATION

Since such a function only performs an action (like printing) and returns nothing, void is the appropriate return type.

Q79 Chapter 8 — Meaning of void

Attempting to use the value of a call to a void function, e.g., x = display();, where display() is void, will:

- **A** Cause a compilation error
- **B** Work normally and store garbage safely
- **C** Always store zero in x
- **D** Convert display() into a non-void function automatically

Ans: A

EXPLANATION

Since a void function returns nothing, assigning its result to a variable is invalid and causes a compile-time error.

Q80 Chapter 8 — Default return type

Modern C standards (such as C99 and later) generally require that:

- **A** No function may specify a return type
- **B** Every function explicitly specify its return type
- **C** Only main() can have a declared return type
- **D** Every function must return type void

Ans: B

EXPLANATION

Later C standards require an explicit return type for every function rather than relying on an implicit int default.

Q81 Chapter 8 — Applications

Which is a typical real-world use of a void function?

- **A** Returning the maximum of two numbers
- **B** Printing a formatted report to the console without returning any data
- **C** Converting Celsius to Fahrenheit and returning the result
- **D** Calculating the square root of a number for later use

Ans: B

EXPLANATION

Since printing does not need to send a value back, a void function is the natural choice for such output-only tasks.

Q82 Chapter 8 Functions returning void

If a function has return type void, which of the following statements can still legally appear inside it?

A return; (with no value)

B return "done";

C return 25;

D return 3.14;

Ans: A

EXPLANATION

A void function may use 'return;' without a value to exit early, but cannot return any actual value.

Chapter 9

9. Passing Values Between Functions Through Interfaces

7 Questions

Q83 Chapter 9 — Parameter passing

The mechanism by which one function sends data to another function it calls is known as:

A Parameter passing
B Compilation
C Type declaration
D Header inclusion

Ans: A

EXPLANATION

Parameter passing is the process of supplying input data (arguments) from a calling function to the called function.

Q84 Chapter 9 — Data communication

Which two mechanisms together allow full two-way data communication between a calling and called function?

A Global variables only
B The file name and the line number
C Parameters (input) and the return statement (output)
D Comments and blank lines

Ans: C

EXPLANATION

Parameters send data into the function, and the return statement sends a result back, enabling two-way communication.

Q85 Chapter 9 — Input/output

When a function needs to both take input and give back a computed output, it should:

A Print all values directly without using a return
B Avoid using return statements
C Accept parameters and use a return statement
D Use only global variables and no parameters

Ans: C

EXPLANATION

This combination — parameters for input, return statement for output — is the standard approach in function design.

Q86 Chapter 9 Function interaction

If function A calls function B and passes it a value, and B computes a result and sends it back, this describes:

A A syntax error in C

B A situation only possible with void functions

C A complete interface-based interaction between two functions

D An example of recursion only

Ans: C

EXPLANATION

This is a normal example of functions interacting through their defined interface — passing values in and receiving a result.

Q87 Chapter 9 Value transfer

In C, when an argument is passed by value, what exactly is transferred to the function?

A The memory address of the argument

B A copy of the argument's value

C The variable's name as a string

D Nothing is transferred; only the function name is passed

Ans: B

EXPLANATION

In call by value, only a copy of the value is passed, not the original variable or its memory address.

Q88 Chapter 9 Data communication

Passing multiple values into a function is achieved by:

A Calling the function multiple times in a row

B Declaring the function as recursive

C Using a single argument that combines all values as text

D Listing multiple arguments, separated by commas, in the function call

Ans: D

EXPLANATION

Multiple values are passed simply by separating them with commas in both the function call and its parameter list.

Q89 Chapter 9 Function interaction

Which of the following correctly reflects value transfer through an interface?

A calculate() modifies a and b directly with no parameters used

B int result; calculate(); — values are transferred without any arguments or return

C int result = calculate(a, b); — values of a and b are passed in, and a value is returned into result

D calculate(a, b); is used but the function has no parameters at all

Ans: C

EXPLANATION

This is a standard example: arguments a and b are passed in, and the returned value is captured in result.

Chapter 10 — 10. Declaration of Function Type — 10 Questions

Q90 Chapter 10 — Declaration

A function declaration (function prototype) in C primarily tells the compiler:

- **A** The name of the programmer who wrote the function
- **B** The function's name, return type, and parameter types before it is used
- **C** The date the program was compiled
- **D** The exact machine code the function will generate

Ans: B

EXPLANATION

A function prototype informs the compiler about the function's signature so it can check calls for correctness before the actual definition appears.

Q91 Chapter 10 — Prototype

Which of the following is a valid function prototype for a function that takes two integers and returns a float?

- **A** void compute(int a, int b) return float;
- **B** float compute(int a, int b);
- **C** float compute;
- **D** compute(int a, int b) { }

Ans: B

EXPLANATION

float compute(int a, int b); correctly declares the return type and parameter types, ending with a semicolon, making it a valid prototype.

Q92 Chapter 10 — Syntax

A function prototype in C must end with a:

- **A** Question mark
- **B** Curly brace
- **C** Colon
- **D** Semicolon

Ans: D

EXPLANATION

Unlike a function definition, a function prototype (declaration) is a statement and must end with a semicolon.

Q93 Chapter 10 Purpose

The main purpose of declaring a function before using it (if defined later in the file) is to:

A Reduce the size of the compiled executable

B Make the program run faster at runtime

C Allow the compiler to perform type checking on function calls

D Automatically generate documentation

Ans: C

EXPLANATION

Declaring the function early lets the compiler verify that calls to the function use the correct number and types of arguments.

Q94 Chapter 10 Declaration rules

If a function is used before it is defined in a C program, without a prior prototype, what could happen (in older/non-strict compilers)?

A The compiler may assume a default return type and not catch parameter mismatches, causing potential errors

B The function is automatically declared as void

C The compiler renames the function automatically

D The program always fails to compile in every C standard

Ans: A

EXPLANATION

Without a prototype, the compiler cannot correctly check the function call, which can lead to incorrect assumptions and runtime issues.

Q95 Chapter 10 Prototype

Which part of a function prototype can be omitted, though it is still good practice to include for readability?

A The parameter names (types must still be specified)

B The semicolon at the end

C The function name

D The return type

Ans: A

EXPLANATION

In a prototype, parameter names are optional (only the types are required by the compiler), though including them aids readability.

Q96 Chapter 10 Purpose

Function declarations are typically placed:

A Inside another unrelated function

B Before main(), or in a header file, so they are visible before the function is called

C Only after the function has already been called

D At the very end of the source file, after all code

Ans: B

EXPLANATION

Prototypes are generally placed before main() or in header files so the compiler knows about them before any calls occur.

Q97 Chapter 10 Syntax

Which of these correctly matches a function prototype to its corresponding definition header?

A int add(int, int); matches void add(int a) { ... }

B int add(int, int); matches int add(char a, char b) { ... }

C int add(int, int); matches float add(int a, int b) { ... }

D int add(int, int); matches int add(int a, int b) { ... }

Ans: D

EXPLANATION

The prototype's return type and parameter types must match the actual function definition exactly (with allowed compatible types).

Q98 Chapter 10 Declaration rules

What is a 'function signature' in the context of a declaration?

A Only the function's return type

B The total number of lines inside the function body

C The programmer's digital signature embedded in the file

D The combination of the function's name and its parameter types

Ans: D

EXPLANATION

A function's signature typically refers to its name together with the types (and order) of its parameters.

Q99 Chapter 10 Declaration

Declaring a function without ever defining it in the program will result in:

A The program running but skipping all function calls silently

B No error at all, since declarations are optional

C A linker error when the function is called

D An automatic empty definition being created

Ans: C

EXPLANATION

If a function is declared and called but never actually defined, the linker cannot find its implementation, causing a linker error.

Chapter 11 — 11. Iterative and Recursive Subprograms — 8 Questions

Q100 Chapter 11 — Iterative

An iterative function solves a problem primarily by using:

- **A** Global variables only
- **B** Repeated calls to itself
- **C** The void keyword
- **D** Loops such as for, while, or do-while

Ans: D

EXPLANATION

Iteration uses looping constructs to repeat a set of steps until a condition is met.

Q101 Chapter 11 — Recursive

A recursive function is one that:

- **A** Is automatically faster than an iterative version
- **B** Never terminates under any circumstance
- **C** Calls itself, directly or indirectly, to solve a smaller instance of the same problem
- **D** Cannot accept any parameters

Ans: C

EXPLANATION

Recursion involves a function invoking itself to break a problem down into smaller sub-problems of the same type.

Q102 Chapter 11 — Characteristics

Which of these is a defining requirement for a correctly written recursive function?

- **A** It must not use any parameters
- **B** It must include a base case to stop the recursion
- **C** It must call a different function, never itself
- **D** It must be declared with return type void

Ans: B

EXPLANATION

Without a base case, a recursive function would call itself indefinitely, leading to infinite recursion and a stack overflow.

Q103 Chapter 11 — Applications

Recursive functions are particularly well-suited to problems that are naturally defined in terms of:

- **A** Only string formatting tasks
- **B** Smaller sub-problems of the same type
- **C** File compression only
- **D** Hardware-level operations exclusively

Ans: B

EXPLANATION

Problems like factorial or Fibonacci are naturally recursive because they can be expressed using smaller versions of themselves.

Q104 Chapter 11 — Advantages

One advantage of using recursion over iteration for certain problems is:

- **A** It never requires a base case
- **B** The code can be simpler and closer to the mathematical definition of the problem
- **C** It eliminates the need for a return statement
- **D** It always uses less memory than iteration

Ans: B

EXPLANATION

For problems with a naturally recursive structure, recursive code can be more elegant and easier to understand than iterative code.

Q105 Chapter 11 — Limitations

A key limitation of recursion compared to iteration is:

- **A** It cannot be used to solve any mathematical problem
- **B** It cannot be written in the C language
- **C** It never produces correct results
- **D** It can consume more memory due to repeated function calls on the call stack

Ans: D

EXPLANATION

Each recursive call adds a new frame to the call stack, which can lead to higher memory usage than an equivalent loop.

Q106 Chapter 11 Comparisons

Which of the following is generally true when comparing iteration and recursion for the same problem?

A There is no practical difference between the two in C

B Iteration is often more memory-efficient, while recursion can be more readable for naturally recursive problems

C Recursion always executes faster than iteration

D Iteration cannot be used to solve any problem recursion can

Ans: B

EXPLANATION

This is a standard comparison: iterative solutions usually use less memory, while recursive solutions can better express certain problems.

Q107 Chapter 11 Applications

Which type of subprogram repeatedly executes a block of code using a loop construct until a condition becomes false?

A A library subprogram

B A recursive subprogram

C An iterative subprogram

D A void subprogram

Ans: C

EXPLANATION

Iterative subprograms use loop constructs (for, while, do-while) to repeat execution based on a condition.

Chapter 12 12. Recursion 11 Questions

Q108 Chapter 12 Concept

Recursion in programming refers to a technique where:

- **A** A variable refers to itself in a declaration
- **B** A function calls itself to solve a problem
- **C** A loop calls another loop
- **D** A program calls another separate program

Ans: B

EXPLANATION

Recursion is defined as a function invoking itself, directly or indirectly, during its own execution.

Q109 Chapter 12 Base case

The base case in a recursive function is important because it:

- **A** Is optional and can always be omitted safely
- **B** Increases the number of recursive calls indefinitely
- **C** Declares all local variables as global
- **D** Provides the condition under which recursion stops

Ans: D

EXPLANATION

The base case is the terminating condition that prevents infinite recursive calls.

Q110 Chapter 12 Recursive call

The statement inside a function where the function invokes itself is referred to as the:

- **A** Return statement
- **B** Local declaration
- **C** Recursive call
- **D** Function prototype

Ans: C

EXPLANATION

The recursive call is the specific line where the function calls itself, typically with a modified/smaller argument.

Q111 Chapter 12 — Execution flow

During recursion, when the base case is finally reached, what happens next?

- **A** The recursion restarts from the very first call
- **B** The entire program terminates immediately
- **C** The function starts returning values back through each previous call on the stack
- **D** All previous calls are deleted without returning any value

Ans: C

EXPLANATION

Once the base case is met, each recursive call returns its result back up the chain of calls, unwinding the stack.

Q112 Chapter 12 — Advantages

A key advantage of recursion is that it can express certain algorithms:

- **A** More naturally and concisely than an equivalent iterative solution
- **B** Only when no parameters are used
- **C** Without ever needing a stopping condition
- **D** With guaranteed lower memory usage than loops

Ans: A

EXPLANATION

For problems like tree traversal or factorial computation, recursive code can closely mirror the problem's mathematical definition.

Q113 Chapter 12 — Limitations

What risk is associated with a recursive function that lacks a proper base case?

- **A** The function will automatically convert to an iterative one
- **B** Infinite recursion, leading to a stack overflow
- **C** The compiler will refuse to compile the program
- **D** The function will return 0 by default and stop safely

Ans: B

EXPLANATION

Without a base case (or an incorrect one), the function keeps calling itself indefinitely, eventually exhausting the call stack.

Q114 Chapter 12 — Applications

Which of the following problems is classically solved using recursion?

- **A** Reading a single character from the keyboard
- **B** Declaring a global constant
- **C** Including a header file
- **D** Calculating the factorial of a number

Ans: D

EXPLANATION

Factorial computation is a classic textbook example of a naturally recursive problem: n! = n * (n-1)!.

Q115 Chapter 12 — Concept

In the recursive definition of factorial, fact(n) = n * fact(n-1), what serves as the base case?

- **A** fact(n) = 0 for all n
- **B** There is no base case needed for factorial
- **C** fact(n) = n * fact(n-1) for all n
- **D** fact(0) = 1 (or fact(1) = 1)

Ans: D

EXPLANATION

The base case fact(0) = 1 (or fact(1) = 1) stops the recursive calls from continuing indefinitely.

Q116 Chapter 12 — Execution flow

Each recursive call in C typically creates a new:

- **A** Global variable shared by all calls
- **B** Separate source code file
- **C** Stack frame (activation record) for its own local variables and parameters
- **D** New copy of the entire program in memory

Ans: C

EXPLANATION

Every function call, including recursive calls, creates its own stack frame containing local variables and parameters.

Q117 Chapter 12 Limitations

Excessive recursion depth without proper termination can lead to which runtime problem?

A Automatic conversion to a library function

B Stack overflow

C Compilation failure

D Infinite loop detection by the compiler

Ans: B

EXPLANATION

Too many nested recursive calls without reaching a base case exhausts the call stack, causing a stack overflow error.

Q118 Chapter 12 Advantages

Recursive solutions are often preferred for problems involving:

A Simple sequential input/output only

B Only single-line arithmetic operations

C Hierarchical or self-similar structures, such as trees

D Programs that never require any function calls

Ans: C

EXPLANATION

Recursion naturally fits problems with hierarchical or repetitive self-similar structures.

Chapter 13 13. Call by Value 12 Questions

Q119 Chapter 13 Definition

In call by value, what is passed to the function?

A Nothing; only the function name is transferred

B The variable name as text

C The memory address of the actual argument

D A copy of the actual argument's value

Ans: D

EXPLANATION

Call by value passes a duplicate of the value stored in the actual argument, not the variable itself.

Q120 Chapter 13 Working

In call by value, if the formal parameter is modified inside the function, what happens to the original variable in the caller?

A It becomes a global variable

B It is deleted from memory

C It remains unchanged

D It is also changed to match the new value

Ans: C

EXPLANATION

Since only a copy is passed, changes to the formal parameter do not affect the original actual argument.

Q121 Chapter 13 Parameter passing

Call by value is the default and most basic method of parameter passing in:

A Only object-oriented languages

B C

C Machine code directly

D Assembly language exclusively

Ans: B

EXPLANATION

C uses call by value as its standard parameter-passing mechanism for ordinary variables.

Q122 Chapter 13 — Memory behavior

In call by value, the formal parameter is stored in:

- **A** A separate memory location from the actual argument
- **B** The operating system's kernel memory
- **C** A read-only section of memory shared by all functions
- **D** The exact same memory location as the actual argument

Ans: A

EXPLANATION

The formal parameter gets its own memory location (typically on the stack), separate from the caller's variable.

Q123 Chapter 13 — Advantages

One advantage of call by value is that it:

- **A** Allows direct modification of the caller's variables
- **B** Protects the original data in the caller from being accidentally modified
- **C** Requires the use of pointers at all times
- **D** Always uses less memory than call by reference

Ans: B

EXPLANATION

Since only a copy is used, the original variable's value is safe from unintended changes inside the function.

Q124 Chapter 13 — Limitations

A limitation of call by value is that it:

- **A** Always causes a compilation error
- **B** Requires the function to be recursive
- **C** Cannot be used to directly modify the caller's original variable
- **D** Cannot be used with more than one parameter

Ans: C

EXPLANATION

Because a copy is passed, call by value alone cannot be used to change the original variable in the calling function.

Q125 Chapter 13 — Applications

Call by value is most appropriate when a function needs to:

- **A** Return multiple values through the parameter list itself
- **B** Change the caller's variable directly
- **C** Use the value of a variable without altering the original
- **D** Access hardware memory addresses directly

Ans: C

EXPLANATION

When a function should only read/use a value without side effects on the caller, call by value is the appropriate choice.

Q126 Chapter 13 — Working

Consider void update(int x) { x = x + 10; } called as update(a);. After the call, the value of 'a' in the caller is:

- **A** Set to 10
- **B** Undefined and unpredictable
- **C** Unchanged, since only a copy of a was modified
- **D** Increased by 10

Ans: C

EXPLANATION

Since call by value passes a copy, modifying x inside update() does not affect the original variable a.

Q127 Chapter 13 — Memory behavior

In call by value, how many independent copies of the data typically exist during the function call?

- **A** Zero copies, since no memory is allocated
- **B** Two: one in the caller and one (the formal parameter) inside the function
- **C** Only one shared copy
- **D** Three copies at minimum

Ans: B

EXPLANATION

The actual argument in the caller and the formal parameter in the function are two separate copies of the value.

Q128 Chapter 13 Definition

Which of these best distinguishes call by value from call by reference?

A Call by value can only be used with arrays

B Both methods are functionally identical in C

C Call by value passes copies of data; call by reference passes access to the original data

D Call by value passes memory addresses; call by reference passes copies

Ans: C

EXPLANATION

This is the fundamental distinction: value passes a copy, reference gives access to the original variable's memory.

Q129 Chapter 13 Advantages

Call by value is generally considered safer for simple data because it:

A Prevents unintended side effects on the caller's variables

B Always executes more slowly than call by reference

C Requires more complex pointer syntax

D Cannot be used with int or float types

Ans: A

EXPLANATION

Because the function only works with a copy, there is no risk of accidentally altering the caller's original data.

Q130 Chapter 13 Limitations

If a function needs to modify multiple variables in the calling function and reflect those changes back, call by value alone is:

A Faster than any alternative in every case

B The only possible solution in C

C Perfectly sufficient with no issues at all

D Insufficient, since it cannot alter the caller's originals

Ans: D

EXPLANATION

Since call by value only works on copies, it cannot be used alone to change and return multiple values back to the caller.

Chapter 14 — 14. Call by Reference — 12 Questions

Q131 Chapter 14 — Definition

Call by reference works by passing:

- **A** The memory address (reference) of the actual argument to the function
- **B** Only a copy of the value, never the address
- **C** The variable's name as a text string
- **D** A duplicate of the entire program

Ans: A

EXPLANATION

In call by reference, the address of the variable is passed, giving the function direct access to the original data.

Q132 Chapter 14 — Working concept

In C, call by reference is typically simulated using:

- **A** The void keyword
- **B** Arrays only, never pointers
- **C** Recursive function calls
- **D** Pointers

Ans: D

EXPLANATION

Since C does not have true reference parameters like some other languages, pointers are used to achieve call-by-reference behavior.

Q133 Chapter 14 — Parameter passing

To pass a variable's address to a function in C, the caller typically uses which operator before the variable name?

- **A** The address-of operator (&)
- **B** The logical NOT operator (!)
- **C** The modulus operator (%)
- **D** The multiplication operator (*)

Ans: A

EXPLANATION

The & operator retrieves the address of a variable, which is then passed to the function to simulate call by reference.

Q134 Chapter 14 Memory behavior

When call by reference is used, modifying the parameter inside the function will:

A Cause the program to crash immediately

B Only affect a local copy, leaving the original unchanged

C Have no effect until the program restarts

D Change the original variable's value in the calling function

Ans: D

EXPLANATION

Because the function operates directly on the original variable's memory address, changes persist after the function returns.

Q135 Chapter 14 Advantages

A key advantage of call by reference is that it allows a function to:

A Modify the caller's original variables directly, including returning multiple results

B Avoid using any parameters at all

C Prevent any changes to variables

D Guarantee that no pointers are used anywhere

Ans: A

EXPLANATION

Since the function can access and change the original data, it can effectively return multiple modified results to the caller.

Q136 Chapter 14 Limitations

A potential limitation or risk of call by reference is that it:

A Always makes programs run slower than call by value

B Can lead to unintended modification of the caller's data if not handled carefully

C Cannot be implemented in the C language at all

D Prevents the function from being called more than once

Ans: B

EXPLANATION

Because the function has direct access to the original variable, careless code can unintentionally alter data the caller did not expect to change.

Q137 Chapter 14 Applications

Call by reference is particularly useful when a function needs to:

A Only display a message on the screen

B Return more than one modified value to the calling function

C Avoid using any return statement or parameters entirely

D Guarantee that the original variable is never touched

Ans: B

EXPLANATION

Since a normal return statement can only send back one value, call by reference (via pointers) allows multiple values to be updated and returned.

Q138 Chapter 14 Working concept

In the function definition void update(int *p) { *p = *p + 1; }, the *p refers to:

A The function's return type

B The address of the pointer variable p itself

C The value stored at the address pointed to by p

D A completely unrelated variable

Ans: C

EXPLANATION

The dereference operator (*) accesses the value stored at the memory address held by pointer p.

Q139 Chapter 14 Parameter passing

To call the function void update(int *p) with a variable named 'count', the correct call is:

A update(count);

B update(count*);

C update(*count);

D update(&count);

Ans: D

EXPLANATION

Since update() expects an address (pointer), the caller must pass the address of count using the & operator.

Q140 Chapter 14 — Comparisons

Compared to call by value, call by reference generally avoids:

- **A** All forms of function calls
- **B** The overhead of copying large amounts of data, since only an address is passed
- **C** The need to ever use a return statement
- **D** The possibility of a function having any parameters

Ans: B

EXPLANATION

Passing an address instead of the full value can be more efficient, especially for larger data such as arrays or structures.

Q141 Chapter 14 — Memory behavior

In call by reference, how many copies of the actual data (not the address) typically exist?

- **A** Only one, since the function accesses the original data directly
- **B** Always exactly two independent copies
- **C** No memory is used at all
- **D** Three or more copies

Ans: A

EXPLANATION

Unlike call by value, call by reference does not duplicate the data itself; only the address is passed, so one copy of the data exists.

Q142 Chapter 14 — Limitations

Which of the following is a common source of errors when using call by reference with pointers?

- **A** Declaring the function with a return type
- **B** Passing too many integer values by mistake
- **C** Using the function more than once in the program
- **D** Dereferencing an uninitialized or invalid pointer

Ans: D

EXPLANATION

A common bug with pointer-based call by reference is dereferencing a pointer that does not point to valid memory, causing undefined behavior.

Chapter 15 15. Return and Their Underlying Implementation 7 Questions

Q143 Chapter 15 Return mechanism

At the implementation level, the return statement typically works by:

A Placing the return value in a designated location (such as a register) and transferring control back to the caller

B Deleting the calling function from memory

C Writing the value directly to a file on disk

D Restarting the program from main()

Ans: A

EXPLANATION

Internally, the return value is often placed in a CPU register, and control is handed back to the point right after the function call.

Q144 Chapter 15 Control transfer

When a function returns, control is transferred back to:

A The point immediately after the function call in the calling code

B The operating system, ending the program

C The very beginning of the program

D A randomly chosen function in the program

Ans: A

EXPLANATION

Execution resumes exactly where the function was called from, right after the call statement.

Q145 Chapter 15 Returning values

In many implementations, where is a function's integer return value commonly placed before control returns to the caller?

A On the hard disk

B In the function's own parameter list

C In a specific CPU register reserved for return values

D In a newly created global array

Ans: C

EXPLANATION

Compilers commonly use a designated CPU register to hold the return value so the caller can retrieve it after the call.

Q146 Chapter 15 Program flow

The return statement affects program flow by:

A Creating a new independent thread of execution

B Ending the current function's execution and resuming the caller's flow

C Preventing the program from ever ending

D Looping back to the start of the same function automatically

Ans: B

EXPLANATION

The return statement is the mechanism that ends a function's flow of execution and resumes the caller's flow.

Q147 Chapter 15 Implementation concepts

The call stack plays a role in return statement implementation by:

A Replacing the need for a return type

B Storing the return address so control can go back to the correct place after the function ends

C Storing only comments from the source code

D Permanently storing all variables from every function ever called

Ans: B

EXPLANATION

The call stack keeps track of the return address (and local data), enabling the program to resume correctly after a function call.

Q148 Chapter 15 Return mechanism

When a function with a non-void return type finishes without an explicit return statement, its underlying behavior is:

A Guaranteed to cause an infinite loop

B Undefined, since no valid return value has been set

C Guaranteed to return the value 1

D Guaranteed to convert the function into void automatically

Ans: B

EXPLANATION

If a non-void function does not execute a return with a value, the value received by the caller is undefined.

Q149 Chapter 15 Control transfer

Underlying the return mechanism, which address is used to determine exactly where execution should resume in the caller?

A The return address stored when the function was called

B The address of the first variable declared in main()

C The address of the operating system kernel

D The address of the last printed output

Ans: A

EXPLANATION

The return address, saved on the call stack at the time of the call, tells the program precisely where to resume in the caller.

Chapter 16 16. Similarities and Comparisons 16 Questions

Q150 Chapter 16 Iteration vs Recursion

Which statement correctly compares iteration and recursion?

A Both can solve repetitive problems, but recursion uses function calls while iteration uses loop constructs

B Recursion never uses the call stack, unlike iteration

C Iteration always requires a base case, just like recursion

D Iteration cannot repeat any task more than once

Ans: A

EXPLANATION

Both techniques repeat work, but they differ in mechanism: recursion via repeated function calls, iteration via loops.

Q151 Chapter 16 Call by Value vs Call by Reference

What is a similarity between call by value and call by reference?

A Both are techniques used to pass arguments to a function

B Both are exclusive to recursive functions

C Both require the use of the void keyword

D Both always modify the caller's original variable

Ans: A

EXPLANATION

Despite their differences in behavior, both are fundamentally mechanisms for passing arguments into a function.

Q152 Chapter 16 Library vs User-defined Functions

Which of the following is true for both library functions and user-defined functions?

A Both can only be used inside the main() function

B Both are invoked using a function call with an appropriate interface

C Both are automatically included without any header file or declaration

D Both must always return an integer value

Ans: B

EXPLANATION

Whether a function is from the standard library or written by the programmer, it is invoked through a function call and interface.

Q153 Chapter 16 — Formal vs Actual Arguments

Which of the following correctly compares formal and actual arguments?

A Actual arguments are supplied at the call site, while formal arguments receive them in the function definition

B Formal and actual arguments always have different data types

C Formal arguments are supplied by the caller directly

D Actual arguments exist only inside the function body

Ans: A

EXPLANATION

Actual arguments are the values given by the caller; formal arguments are the corresponding parameters that receive them inside the function.

Q154 Chapter 16 — Return Value vs void

How do return-value functions and void functions differ in their interface?

A Return-value functions cannot accept any parameters

B Void functions always accept more parameters than return-value functions

C There is no meaningful difference between them

D Return-value functions send a result back via return; void functions do not send back any value

Ans: D

EXPLANATION

The key difference lies in the return type: one produces an output value, and the other does not.

Q155 Chapter 16 — Local Variables vs Function Parameters

What do local variables and function parameters have in common?

A Both exist for the entire duration of the program by default

B Both must be declared using the keyword void

C Both are typically scoped to, and exist only within, the function in which they are declared

D Both are always initialized to the same default value

Ans: C

EXPLANATION

Function parameters behave much like local variables — both are limited in scope and lifetime to the function itself.

Q156 Chapter 16 Function Declaration vs Function Definition

How does a function declaration differ from a function definition?

A A declaration includes the function body, while a definition does not

B They are exactly the same and interchangeable in every context

C A definition can never include a return type

D A declaration specifies only the interface (prototype), while a definition provides the full body of the function

Ans: D

EXPLANATION

A declaration (prototype) tells the compiler about the interface; the definition provides the actual implementation/body.

Q157 Chapter 16 Iteration vs Recursion

In terms of memory usage, how do iterative and recursive solutions typically compare for the same problem?

A Recursive solutions always use less memory than iterative ones

B Iterative solutions generally use less memory since they avoid repeated function call overhead

C Both use exactly the same amount of memory in all cases

D Neither approach uses any memory at all

Ans: B

EXPLANATION

Because recursion adds a new stack frame per call, it commonly uses more memory than an equivalent loop-based solution.

Q158 Chapter 16 Call by Value vs Call by Reference

Which comparison between call by value and call by reference is accurate?

A Call by value is implemented using the & operator in C

B Call by value protects the original data; call by reference allows direct modification of the original data

C Call by reference always uses more memory to store full data copies

D Both methods behave identically in every situation

Ans: B

EXPLANATION

This is the core practical distinction between the two parameter-passing methods.

Q159 Chapter 16 Library vs User-defined Functions

Which is a key difference between library functions and user-defined functions?

A Library functions are pre-written and provided by the compiler/standard library, while user-defined functions are written by the programmer

B Library functions must always be recursive

C User-defined functions cannot accept parameters

D There is no difference; they are the same in C

Ans: A

EXPLANATION

Library functions come ready-made with the compiler (e.g., in stdio.h), whereas user-defined functions are created by the programmer for specific tasks.

Q160 Chapter 16 Formal vs Actual Arguments

Formal parameters and actual arguments must generally agree in which aspect during a function call?

A The number of return statements in the function

B The order in which functions are defined in the file

C Number of arguments and compatible data types

D Variable names used in each function

Ans: C

EXPLANATION

For a function call to work correctly, the actual arguments must correspond in count and type to the formal parameters.

Q161 Chapter 16 Return Value vs void

Which scenario would call for a void function rather than one with a return value?

A When the caller needs to store the function's result in a variable

B When the function is being used purely for a mathematical calculation

C When the function must send back a calculated total

D When the function only needs to perform an action, such as displaying output, without producing a usable result

Ans: D

EXPLANATION

Void functions are appropriate for action-only tasks where no value needs to be communicated back to the caller.

Q162 Chapter 16 Iteration vs Recursion

Which of these problems is typically easier to express using recursion rather than plain iteration?

A Traversing a hierarchical, tree-like structure

B Reading a single value from the keyboard

C Adding two integers together

D Printing numbers from 1 to 10 in sequence

Ans: A

EXPLANATION

Hierarchical/self-similar structures, like trees, are naturally suited to a recursive approach compared to simple iteration.

Q163 Chapter 16 Local Variables vs Function Parameters

Which statement correctly distinguishes function parameters from ordinary local variables?

A Parameters can never be modified inside the function

B Parameters do not occupy any memory, unlike local variables

C Local variables are always visible to other functions, unlike parameters

D Parameters receive their initial values from the caller, while ordinary local variables are typically initialized within the function body

Ans: D

EXPLANATION

The key distinction is how each gets its initial value: parameters come from the caller, while local variables are set within the function.

Q164 Chapter 16 Call by Value vs Call by Reference

Assertion (A): Call by reference is implemented in C using pointers. Reason (R): C does not support true reference parameters like some other languages. Choose the correct option.

A A is false, but R is true

B Both A and R are false

C A is true, but R is false

D Both A and R are true, and R correctly explains A

Ans: D

EXPLANATION

Since C lacks native reference parameters, pointers are used to simulate call by reference — R correctly explains why A is true.

Q165 Chapter 16 Function Declaration vs Function Definition

Assertion (A): A function can be declared multiple times but defined only once in a program. Reason (R): Multiple declarations simply repeat the same interface information for the compiler, while multiple definitions would create conflicting implementations. Choose the correct option.

A Both A and R are true, and R correctly explains A

B A is true, but R is false

C A is false, but R is true

D Both A and R are false

Ans: A

EXPLANATION

A function's prototype can appear multiple times without conflict, but having more than one definition (body) causes a redefinition error — which R explains correctly.